

基于决策树及集成算法的回归与分类案例

一 任务描述

本任务基于决策树及集成算法分别实现鲍鱼年龄预测案例和肿瘤分类案例。鲍鱼年龄预测案例是建立一个回归模型，根据鲍鱼的特征数据（长度、直径、高度、总重量、剥壳重量、内脏重量、壳重）等预测其年龄。

肿瘤分类案例我们已经用逻辑回归实现过，本任务利用决策树和集成算法来实现。

本任务的主要实践内容：

- 1、决策树分类模型的构建
- 2、决策树算法的调参
- 3、集成算法（随机森林、Adaboost）的应用及对比
- 4、使用matplotlib可视化模型预测曲线

二 任务目标

- | | |
|-------------------------|-----|
| 1. 基于决策树和随机森林构建鲍鱼年龄预测模型 | 重点 |
| 2. 基于决策树及集成算法构建肿瘤预测模型 | 重点 |
| 3. 理解决策树算法的原理 | 重难点 |
| 4. 理解决策树算法的主要参数（剪枝参数） | 重难点 |
| 5. 掌握集成算法的使用 | 重难点 |

三 任务环境

- 操作系统：Windows 10、Ubuntu18.04
- 工具软件：Anaconda3 2019、Python3.7
- 硬件环境：无特殊要求
- 依赖库

```
matplotlib 3.3.4
numpy 1.19.5
pandas 1.1.5
scikit-learn 0.24.2
mglearn 0.1.9
```

四 任务分析

鲍鱼年龄预测模型的输出（年龄）是个连续值，因此这是一个回归问题，算法的目的是寻找鲍鱼的特征数据和年龄之间的规律（即回归函数）。

本任务涉及以下几个环节：

- a) 加载、查看数据集
- b) 数据的处理及拆分
- d) 构建模型，拟合训练数据
- e) 评估并预测鲍鱼年龄

子任务：基于决策树与集成算法，构建分类模型预测肿瘤分类

五 任务实施

5.1 加载、查看鲍鱼数据集

Abalone数据集可以在线加载，但建议下载到本地，然后加载本地文件（方法2）。

```
import numpy as np
import pandas as pd
from IPython.display import display

# 方法1、在线加载鲍鱼数据集文件（csv格式，建议复制链接下载到本地，然后使用方法2加载）
# url = 'http://archive.ics.uci.edu/ml/machine-learning-
#       databases/abalone/abalone.data'
# data = pd.read_csv(url) # 读取在线数据

# 方法2、加载本地数据文件（abalone.data文件放在当前目录）
abalone = pd.read_csv('../dataset/abalone.data')

# 原数据中没有表头（即特征名称），我们加上便于观察数据
abalone.columns = ['Sex', 'Length', 'Diameter', 'Height',
                   'Whole weight', 'Shucked weight', 'Viscera weight',
                   'Shell weight', 'Rings']

display(abalone) # 4176条数据，8个特征（最后一列Rings代表环数，即年龄，相当于数据的标签）
```

显示结果：

	Sex	Length	Diameter	Height	Whole weight	Shucked weight	Viscera weight	Shell weight	Rings
0	M	0.350	0.265	0.090	0.2255	0.0995	0.0485	0.0700	7
1	F	0.530	0.420	0.135	0.6770	0.2565	0.1415	0.2100	9
2	M	0.440	0.365	0.125	0.5160	0.2155	0.1140	0.1550	10
3	I	0.330	0.255	0.080	0.2050	0.0895	0.0395	0.0550	7
4	I	0.425	0.300	0.095	0.3515	0.1410	0.0775	0.1200	8
...	...	...	...	...	...	...	...	...	...
4171	F	0.565	0.450	0.165	0.8870	0.3700	0.2390	0.2490	11
4172	M	0.590	0.440	0.135	0.9660	0.4390	0.2145	0.2605	10
4173	M	0.600	0.475	0.205	1.1760	0.5255	0.2875	0.3080	9
4174	F	0.625	0.485	0.150	1.0945	0.5310	0.2610	0.2960	10
4175	M	0.710	0.555	0.195	1.9485	0.9455	0.3765	0.4950	12

4176 rows × 9 columns

数据

类别

4176条数据，8个特征，最后一列Rings代表年龄，相当于数据的标签（target）。

5.2 数据处理及拆分

前几节里我们用的数据集都是scikit-learn自带的，内部已经将data（样本特征数据）和target（标签）分开，而鲍鱼数据集需要我们自己分开，然后才能使用train_test_split函数将data和target拆分为训练集和测试集。

另外，需要注意的是，性别（Sex）列不是数字，而是M（雄性）、F（雌性）和I（未知）三个字符，为了便于模型处理，我们需要将其转换为数字，这里我们用0、1、2分别替换M、F和I。

```
# 性别特征的值是M、F和I（未知），为了便于算法计算，需要变成数字
new_abalone = abalone.replace({'M':0, 'F':1, 'I':2}) # M、F、I分别替换为0,1,2
display(new_abalone)
```

显示结果：

	Sex	Length	Diameter	Height	Whole weight	Shucked weight	Viscera weight	Shell weight	Rings
0	0	0.350	0.265	0.090	0.2255	0.0995	0.0485	0.0700	7
1	1	0.530	0.420	0.135	0.6770	0.2565	0.1415	0.2100	9
2	0	0.440	0.365	0.125	0.5160	0.2155	0.1140	0.1550	10
3	2	0.330	0.255	0.080	0.2050	0.0895	0.0395	0.0550	7
4	2	0.425	0.300	0.095	0.3515	0.1410	0.0775	0.1200	8
...	...	...	...	...	...	...	...	...	...
4171	1	0.565	0.450	0.165	0.8870	0.3700	0.2390	0.2490	11
4172	0	0.590	0.440	0.135	0.9660	0.4390	0.2145	0.2605	10
4173	0	0.600	0.475	0.205	1.1760	0.5255	0.2875	0.3080	9
4174	1	0.625	0.485	0.150	1.0945	0.5310	0.2610	0.2960	10
4175	0	0.710	0.555	0.195	1.9485	0.9455	0.3765	0.4950	12

接下来，我们获取前8列作为data（样本特征数据）、最后1列年龄作为target（样本的标签），然后再使用train_test_split函数将data和target随机拆分为训练集和测试集。

```

from sklearn.model_selection import train_test_split

# 从数据集中分离样本数据和标签数据
data = new_abalone.values[:, :8] # 前8列（特征数据）
target = new_abalone.values[:, -1] # 最后一列（年龄）

# 拆分数据集
X_train, X_test, y_train, y_test = train_test_split(data, target,
                                                    test_size=0.25,
                                                    random_state=0)

print(X_train.shape, X_test.shape) # 查看拆分结果

```

输出结果：

```
(3132, 8) (1044, 8)
```

5.3 创建模型，评估与预测

我们使用三种模型来建模，分别是：决策树（默认参数）、随机森林（集成50棵决策树）和线性回归。

```

from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import LinearRegression

# 创建回归模型
# model = DecisionTreeRegressor().fit(X_train, y_train) # 决策树（默认参数）
model = RandomForestRegressor(50).fit(X_train, y_train) # 使用50棵决策树构成随机森林
model = LinearRegression().fit(X_train, y_train) # 线性回归

# 评估模型（分类模型输出准确率，回归模型输出R2_score）
print('test_score:', model.score(X_test, y_test))

# 预测测试集中的鲍鱼年龄
y_pred = model.predict(X_test)
n = 10 # 显示前n个样本的预测年龄，并与实际年龄作对比
print('预测年龄: ', np.round(y_pred[:n])) # np.round() - 四舍五入取整
print('实际年龄: ', y_test[:n])

```

显示结果：

```
# 决策树（默认参数）
test_score: 0.46037735849056594
预测年龄: [0. 1. 1. 1. 1. 1. 1. 1. 1. 1.]
实际年龄: [0 1 1 1 1 1 1 1 1 1]
# 使用50棵决策树构成随机森林
test_score: 0.8677924528301887
预测年龄: [0. 1. 1. 1. 1. 1. 1. 1. 1. 1.]
实际年龄: [0 1 1 1 1 1 1 1 1 1]
# 线性回归
test_score: 0.7291758706114061
预测年龄: [0. 1. 1. 1. 1. 1. 1. 1. 1. 1.]
实际年龄: [0 1 1 1 1 1 1 1 1 1]
```

可以看到：使用默认参数的决策树回归算法表现最差，使用50棵决策树集成的随机森林和逻辑回归模型表现要好一些。

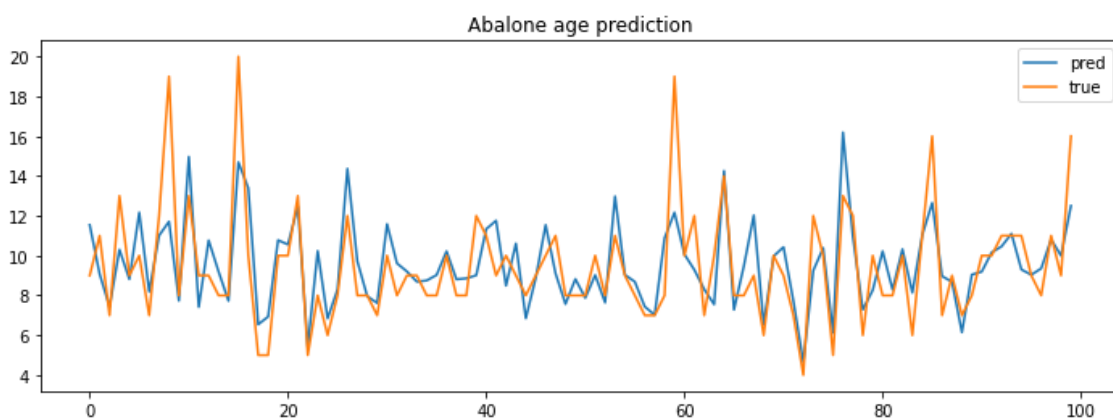
5.4 可视化预测曲线

为了形象地观察模型预测的年龄和实际年龄的关系，我们使用Matplotlib将预测值和实际值可视化。

```
import matplotlib.pyplot as plt

# 可视化预测曲线
plt.figure(figsize=(12, 4)) # 图像尺寸
plt.title('Abalone age prediction') # 标题
n = 100 # 图中显示样本的数量
plt.plot(np.arange(n), y_pred[:n], label='pred') # 预测值
plt.plot(np.arange(n), y_test[:n], label='true') # 实际值
plt.legend() # 显示图例
plt.show()
```

显示结果：



5.5 子任务：使用多种模型实现肿瘤预测（分类问题）

决策树及其集成算法既可以处理回归问题，也可以处理分类问题，下面对比它们在肿瘤分类预测中的表现。

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split

cancer = load_breast_cancer() # 加载cancer数据集

# 拆分数据集
X_train, X_test, y_train, y_test = train_test_split(cancer.data, cancer.target,
                                                    test_size=0.25,
                                                    random_state=0)
# 创建多种分类模型
m_1 = DecisionTreeClassifier().fit(X_train, y_train) # 无参数决策树（过拟合现象严重）
m_2 = DecisionTreeClassifier(max_depth=5, min_samples_leaf=9).fit(X_train,
y_train) # 指定剪枝参数
m_3 = RandomForestClassifier(50).fit(X_train, y_train) # 随机森林（使用50棵决策树）
m_4 = AdaBoostClassifier().fit(X_train, y_train) # AdaBoost

# 评估模型，对比准确率
models = {'d_tree_1':m_1, 'd_tree_2':m_2, 'r_forest':m_3, 'adaboost':m_4}
for m in models:
    score_train = models[m].score(X_train, y_train)
    score_test = models[m].score(X_test, y_test)
    print('{}: train_score {:.2f} test_score {:.2f}'.format(m, score_train,
score_test))
```

输出结果：

```
d_tree_1: train_score 1.00 test_score 0.87
d_tree_2: train_score 0.96 test_score 0.94
r_forest: train_score 1.00 test_score 0.97
adaboost: train_score 1.00 test_score 0.98
```

可以看出，无参数的决策树模型（d_tree_1）过拟合现象比较严重，而指定了剪枝参数的决策树（d_tree_2）和两个决策树集成模型（随机森林、AdaBoost）表现更好。

六 任务总结

- 本实验使用决策树、随机森林和线性回归分别来预测鲍鱼年龄和肿瘤分类，随机森林的表现最好。
- 决策树及其集成算法既可以处理回归问题，也可以处理分类问题。

七 任务知识测试

1、决策树算法的最主要缺点是：（单选）（ ）

- A、容易欠拟合
- B、容易过拟合
- C、效率不高

2、下列属于决策树集成算法的是：（多选）（ ）

- A、随机森林
- B、AdaBoost
- C、逻辑回归
- D、线性回归

3、哪些方法可以降低决策树的过拟合现象：（多选）（ ）

- A、剪枝
- B、使用集成算法
- C、增加样本数量

4、下列属于决策树算法参数的是：（多选）（ ）

- A、max_depth
- B、min_samples_leaf
- C、min_samples_split
- D、random_state